Useful Commands and Features of Matlab

1. Array-handling capabilities

Matlab's ability to treat vectors and arrays as objects can (and should) be used to avoid loops. For example, a series of angles from 0 to 6π in 0.02 increments is generated by the command

```
a2 = 0:0.02:6*pi; % where the semicolon prevents screen output.
```

Now that we have `a2` (as a row vector), we can create any function of `a2` without using a loop:

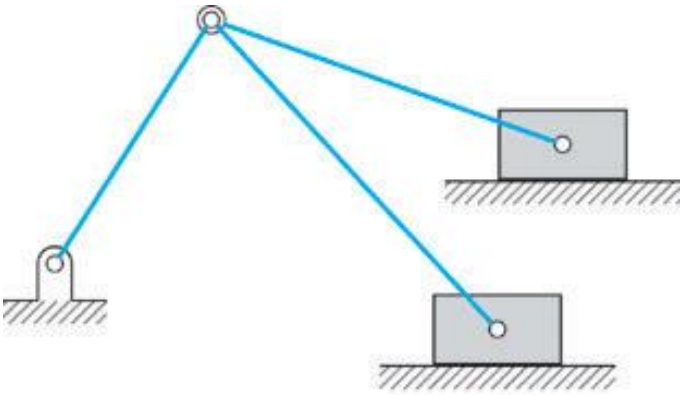
```
f = 5*cos(a2)+1; % gives a vector the same length as a2.
```

The cosine function is applied point-wise (i.e. element-by-element) to a2, as is the factor of 5 and the addition of 1. However, if there can be any confusion with matrix operations, you need to use to use point-wise notation explicitly:

```
f = (sin(a2).*cos(a2)).^2; % gives a vector the same length as a2.
```

Here, point-wise multiplication .* is necessary: if we omit the point, Matlab will attempt matrix multiplication (which will fail since sin(a2) and cos(a2) are both 1xn). Similarly, the operator .^2 squares the resulting vector point-wise instead of attempting to square it as a 1xn matrix using ^2 (without the point).

2. Animations and Plotting Commands



(b)

You will be required to animate mechanisms, such as the double slider crank shown above. One animation technique is to create an array of movie frames and then play it using the movie command; you are welcome to try this. Here I will give some tips on a slightly simpler approach, which is to update the object positions in the figure window after a suitable pause. You should do all position calculations outside of this plotting loop (using the above array techniques) so that you are not slowing down the plotting loop and also to make your program more modular.

One way to update object positions is to use the `hold on` and `hold off` commands to indicate whether you are adding objects to the plot or overwriting (deleting) the existing ones. This approach requires that you set the axis limits (using the `axis` command) every time the loop executes, otherwise these limits (and the plotted objects) will jump around.

A better way is to use the `set` command, which can be used to change the position coordinates of named objects that you have already plotted. Consider the following sequence:

```
figure(1) % set figure 1 as the current figure.
% Plot 2 links (lines) and 3 pin joints (o markers):
links = plot([0 x1(0) x2(0)], [0 y1(0) y2(0)],'-o','linewidth',2);
% Plot a slider on the ground:
slider = rectangle('Position',[x(0),y,w,h], 'linewidth', 1);
ground = line([x-1, x+1], [y,y], 'linewidth', 2);


axis equal % equalize the x and y axes so that objects are not stretched.
axis([left, bottom, width, height]); % set axis limits

% Update the plot
for n = 2:length(a2)
    pause(0.1) % Pause for 0.1 seconds
    set(links,'XData',[0 x1(n) x2(n)],'YData',[0 y1(n) y2(n)])
    set(slider, 'Position',[x(n),y,w,h])
end
```

In the initial plot, the variable names `links` and `slider` are assigned handles (numerical codes) that are referenced inside the plot update loop to change their data with each time step. Feel free to use different colours for different links, etc. Note that the above code is not actual code for Assignment 1; you will need to work that out yourself. Also, please be reminded that each submission should be unique.

3. Miscellaneous Command Help

Besides the animation, Assignment 1 asks you to plot 4 curves of slider speed on the same plot. The `hold on` and `hold off` commands are useful for this. Please look up the `diff` command to facilitate numerical differentiation. Again, you do not need any loops to do these computations or to plot speed.

You can look up the syntax of any command by typing `help command_name` at the command prompt or by searching the online help. If you are rusty on Matlab, try reviewing the online tutorials.